

CAD-Coder: Scaling and Deploying Vision-Language Models for CAD Code Generation

Authors: Yuyan Fan (yuyan_fan@brown.edu), Jing Xu (jing_xu3@brown.edu), Frank Chen (sic963@g.harvard.edu), and Aditya Palaparthi (aditya_palaparthi@g.harvard.edu)

Background and Motivation: Computer-Aided Design (CAD) is essential to engineering and manufacturing, yet creating CAD models remains slow and expertise-driven. Recent advances in vision-language models (VLMs) offer a chance to accelerate and democratize CAD workflows. Our project builds on [CAD-Coder](#), an open-source model that converts text and images into CadQuery Python code. While promising, CAD-Coder has not been systematically evaluated across model scales or deployed in an accessible interface. We will close this gap by fine-tuning scaled variants, analyzing performance trends, and deploying them in a chatbot that enables engineers, students, and hobbyists to generate CAD code interactively. Compared to manual modeling, proprietary CAD tools, or limited AI plugins, our system lowers barriers to entry, speeds iteration, and contributes new insights into multimodal model scaling. Our team brings strong data science expertise, including research experience at MIT on generative AI and 3D engineering design, positioning us well to execute this project.

Problem Statement: We aim to design and deploy a scalable chatbot system that generates CAD code from text or image inputs, while studying how performance scales across model sizes.

Scope and Objectives: Our project scope is to deploy a research-informed CAD chatbot. Namely:

1. **Scaling Analysis**
 - Fine-tune [Qwen-2.5](#) models (3B, 7B, 14B, 32B, 72B) on the GenCAD dataset and analyze scaling trends in accuracy, efficiency, and robustness.
2. **Deployment**
 - Deploy two models on Google Cloud VertexAI: Original CAD-Coder (LLaVA baseline) and best fine-tuned Qwen-2.5 variant
3. **Chatbot Interface**
 - Build a **ChatGPT-style interface** with text input + image upload, model selection dropdown, chat history, and Retrieval-Augmented Generation (RAG)
4. **Efficient Inference**
 - Implement optimized pipelines (quantization, FlashAttention, continuous batching, multi-GPU scheduling) to ensure practical latency and scalability.

Users will include **mechanical engineers** for rapidly prototyping CAD components, **students & educators** for having an accessible entry point for learning CAD, and **makers & hobbyists** for generating parts without mastering complex CAD software. **Benefits over Alternatives:** Unlike commercial CAD tools or plugins, our system automates code generation directly from natural inputs (text/images), provides open, reproducible research into model scaling, and offers a flexible, cloud-deployed interface for broader accessibility.

Data Sources: We use the **GenCAD dataset** (163k image-code pairs of CAD models with CadQuery scripts) from MIT's CAD-Coder project (link: [CADCODER/GenCAD-Code · Datasets at Hugging Face](#)). This dataset provides the exact multimodal alignment needed for text-to-code and image-to-code

generation. Its large scale and diversity enable both rigorous scaling analysis and deployment of models that generalize to real CAD design tasks.

- **Features:** 2D rendered images, paired CAD code, metadata.
- **Coverage:** Basic geometric primitives (cubes, cylinders) to assemblies (frames, chassis).
- **Advantages:** Larger and more diverse than comparable CAD datasets.
- **Challenges:** Low-resolution or synthetic-looking images may hinder generalization to real-world sketches. We will implement preprocessing (resizing, normalization, augmentation) to improve robustness.

Preliminary Design: System components will include a **frontend** for a chatbot with multimodal input and model selection, a **backend** for VertexAI-managed inference pipelines for scalable serving, and **CI/CD** for deployment with Docker + Kubernetes. Our **mockup is a ChatGPT-style chat window** with file upload, dropdown to switch models, and RAG-backed history. Here is the link to our public Figma design: <https://casual-lion-94885387.figma.site/>

Research Foundations will be grounded in several research areas. **Scaling laws** ([Kaplan et al.](#), [Hoffmann et al.](#)) provide the theoretical basis for analyzing performance trends across Qwen-2.5 model sizes, guiding expectations about efficiency and accuracy trade-offs. **Efficient fine-tuning methods** such as [LoRA](#) and [OLoRA](#) enable practical adaptation of large models within resource constraints, making experiments with models up to 72B parameters feasible. Finally, **multimodal instruction tuning frameworks** like [LLaVA](#) and [LLaVA-Next](#) establish effective techniques for aligning visual and textual modalities, which directly informs our approach to fine-tuning CAD-Coder on the GenCAD dataset.

Milestones

- **10/10** – Baseline CAD-Coder setup
- **10/16** – Fine-tune and evaluate Qwen-2.5 models
- **10/28** – Data preprocessing & OOD pipeline
- **11/10** – Backend inference pipeline + RAG integration
- **11/17** – Frontend development (chat UI)
- **11/25** – CI/CD deployment + chatbot integration
- **12/11** – Final testing, demo, and report

Risks and Limitations

- **Generalization:** Training on clean synthetic CAD images may limit performance on messy real-world sketches.
- **Resource Costs:** Fine-tuning large models ($\geq 7B$ parameters) is compute-intensive, but we have access to Aditya Palaparthi's H100/200 cluster from his MIT lab.
- **Latency/Cost Tradeoff:** Serving very large models in real time requires aggressive optimization.
- **Infrastructure Complexity:** Managing multi-GPU and cloud pipelines adds deployment challenges.

Fun Factor: The chatbot transforms design ideas (images) into executable CAD code in real time. The instant feedback loop of seeing designs generated directly from prompts makes this both technically challenging and highly engaging.